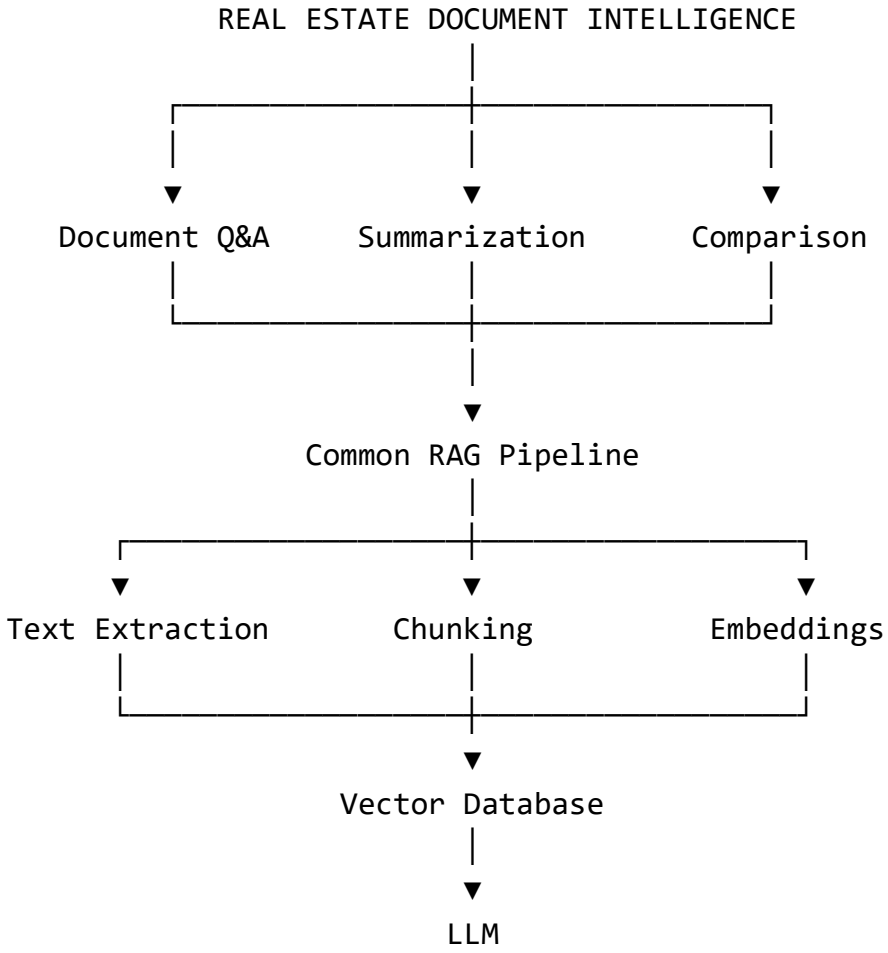
1. Think of the project like this



So the **bottom part is common**, while the **top three parts are your features**.

2. Your existing structure

You currently have:

```
real_estate_rag/
│
├── app.py
├── requirements.txt
├── .env
```

```
│
├── documents/
│
├── vector_db/
│
├── rag/
│   ├── document_loader.py
│   ├── text_splitter.py
│   ├── embeddings.py
│   ├── vector_store.py
│   ├── retriever.py
│   ├── prompts.py
│   └── comparison.py
│
└── utils/
    └── preprocessing.py
```

This is okay for the **RAG foundation**, but I would modify it slightly so your three features are clearly represented.

3. Recommended final structure

I recommend this:

```
real_estate_rag/
│
├── app.py
├── requirements.txt
├── .env
│
├── documents/
│   │
│   ├── property_A/
│   │   ├── property_details.pdf
│   │   ├── rental_agreement.pdf
│   │   └── inspection_report.pdf
│   │
```

```
│   └── property_B/
│       ├── property_details.pdf
│       ├── rental_agreement.pdf
│       └── inspection_report.pdf
│
├── vector_db/
│
├── rag/
│   │
│   ├── document_loader.py
│   ├── text_splitter.py
│   ├── embeddings.py
│   ├── vector_store.py
│   ├── retriever.py
│   └── prompts.py
│
├── features/
│   │
│   ├── qa.py
│   ├── summarization.py
│   └── comparison.py
│
└── utils/
    └── preprocessing.py
```

Now it is much easier to understand.

4. What each folder does

rag/

This contains your **common RAG pipeline**.

```
rag/
├── document_loader.py
├── text_splitter.py
├── embeddings.py
```

```
├── vector_store.py
├── retriever.py
└── prompts.py
```

These are shared by all three features.

5. features/

This contains your actual **GenAI features**.

```
features/
│
├── qa.py
├── summarization.py
└── comparison.py
```

This is the important change.

6. Feature 1 — Document Q&A

File:

`features/qa.py`

This handles questions such as:

What is the security deposit?

What is the monthly rent?

What is the property area?

Who is responsible for maintenance?

The flow is:

```
User Question
      ↓
Retriever
      ↓
Relevant Chunks
      ↓
LLM
      ↓
Answer
```

Example:

```
User:
What is the security deposit?

        ↓

Retriever:
"Security deposit shall be ₹1,00,000."

        ↓

LLM:

The security deposit is ₹1,00,000.
```

Important

This feature **uses RAG** because you don't want the LLM to guess the answer.

7. Feature 2 — Document Summarization

File:

`features/summarization.py`

This handles:

Summarize this agreement.

The flow is slightly different.

```
Selected Document
       ↓
Extract Text
       ↓
Chunks
       ↓
LLM
       ↓
Summary
```

For example:

```
Rental Agreement
        ↓
Important information
        ↓
LLM
        ↓
Summary
```

Output:

```
Agreement Summary
```

- Property: 2 BHK apartment
- Area: 1200 sq.ft
- Monthly Rent: ₹25,000
- Security Deposit: ₹1,00,000
- Lease Period: 11 months
- Notice Period: 2 months
- Maintenance: Tenant handles minor repairs

Does summarization use RAG?

It can, but it doesn't necessarily need retrieval.

For a single short document, you can directly send the document text to the LLM.

For a long document, you can use:

```
Document
 ↓
Chunks
 ↓
Summarize chunks
 ↓
Combine summaries
 ↓
Final summary
```

This is usually called **map-reduce/document summarization** rather than normal question-answering RAG.

So:

Feature	Needs Retrieval?
Document Q&A	✅ Yes
Summarization	⚠️ Not always
Property Comparison	✅ Yes

8. Feature 3 — Multi-document Comparison

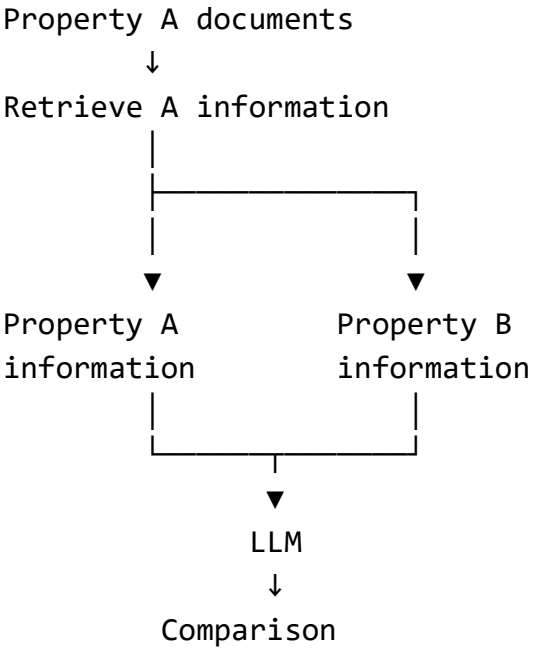
File:

`features/comparison.py`

This handles:

Compare Property A and Property B.

Now we have:



For example:

```
Property A
Area: 1200 sq.ft
Rent: ₹25,000
Deposit: ₹1,00,000

Property B
Area: 1500 sq.ft
Rent: ₹30,000
Deposit: ₹1,50,000
```

The LLM produces:

Feature	Property A	Property B
Area	1200 sq.ft	1500 sq.ft
Rent	₹25,000	₹30,000
Deposit	₹1,00,000	₹1,50,000

Then:

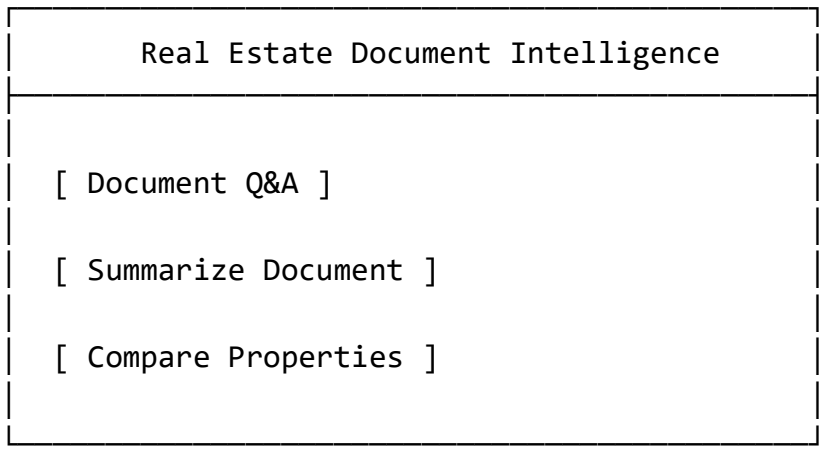
Property B has a larger area, but it also has a higher monthly rent and security deposit.

9. How does the system know which feature the user wants?

This is an important question.

Your frontend can make it very simple.

Instead of trying to automatically understand everything initially, give the user three options:



Then each button calls a different backend endpoint.

10. Your Flask API can identify the feature

For example:

Q&A

`POST /api/qa`

Request:

```
{
    "question": "What is the security deposit?"
}
```

Summarization

```
POST /api/summarize
```

Request:

```
{
    "document": "rental_agreement.pdf"
}
```

Comparison

```
POST /api/compare
```

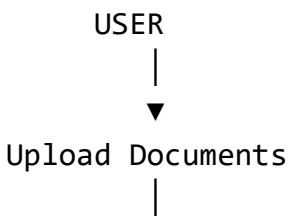
Request:

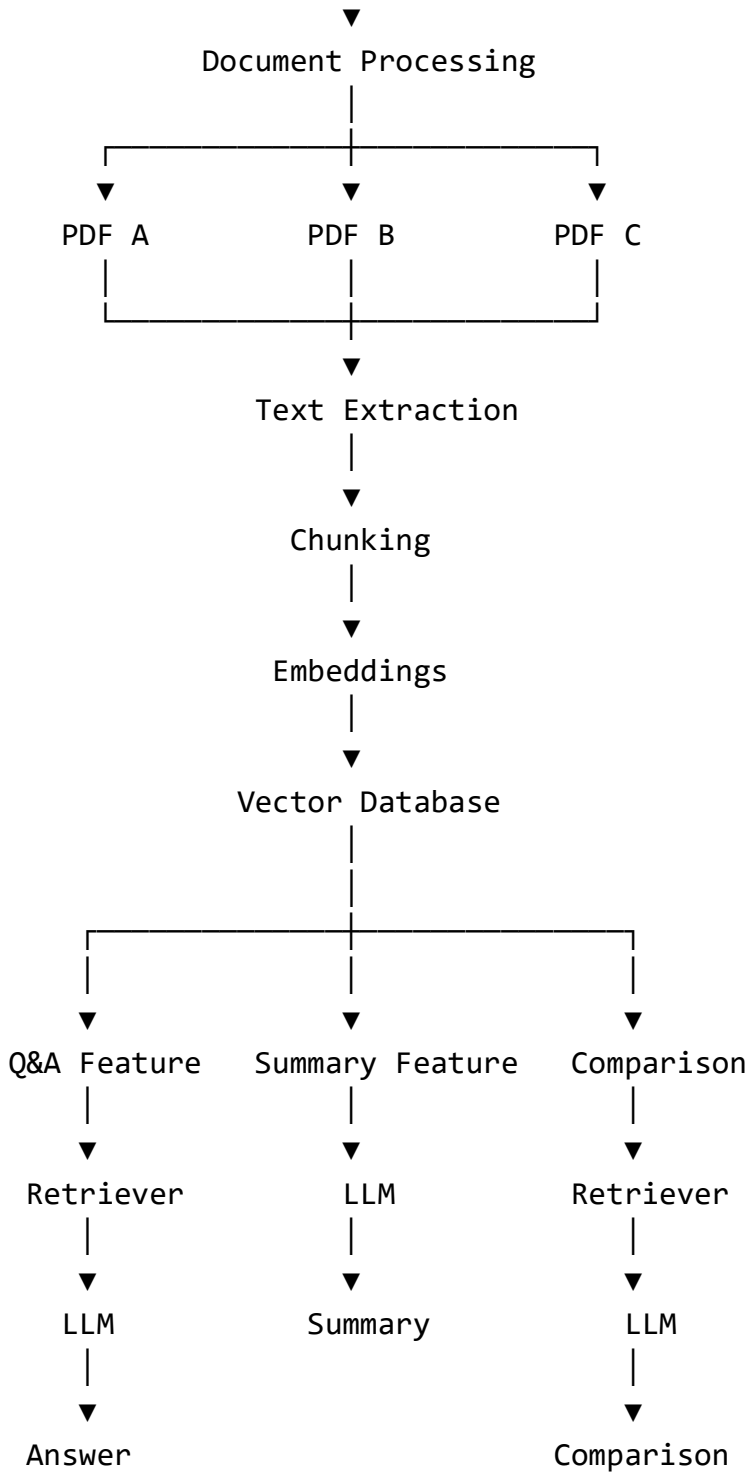
```
{
    "property_a": "property_A",
    "property_b": "property_B"
}
```

This makes your architecture very clear.

11. Complete project flow

Your final system will work like this:




Document Processing
PDF A
PDF B
PDF C
Text Extraction
Chunking
Embeddings
Vector Database
Q&A Feature
Summary Feature
Comparison
Retriever
LLM
Retriever
LLM
Summary
LLM
Answer
Comparison

12. Your documents/ folder is also important

You asked earlier whether you need to create Property A and Property B files.

Yes, but **you don't need Property A and B for the first stages**.

I recommend developing in this order.

Phase 1 — Document Q&A

Start with only:

```
documents/
└── rental_agreement.pdf
```

Build:

```
PDF
 ↓
Text
 ↓
Chunks
 ↓
Embeddings
 ↓
Vector DB
 ↓
Question
 ↓
Retriever
 ↓
LLM
 ↓
Answer
```

Make sure:

What is the security deposit?

works.

13. Then add summarization

Keep the same document:

```
documents/
└── rental_agreement.pdf
```

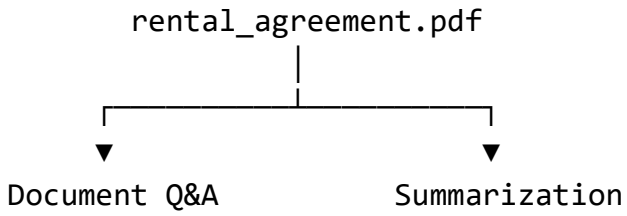
Add:

```
features/
├── qa.py
└── summarization.py
```

Test:

Summarize this agreement.

Now you have:



14. Only after that add Property A and B

Then change:

```
documents/
└── rental_agreement.pdf
```

to:

```
documents/
│
├── property_A/
│   └── rental_agreement.pdf
│
└── property_B/
    └── rental_agreement.pdf
```

Then add:

```
features/
├── qa.py
├── summarization.py
└── comparison.py
```

Now your project has all three capabilities.

15. The development order I recommend for YOU

Don't start with Property A and Property B immediately.

Follow this exact order:

Phase 1 — Single document

```
PDF
 ↓
Extract text
 ↓
Chunks
 ↓
Embeddings
 ↓
Vector DB
```

Phase 2 — Q&A

```
Question
 ↓
Retriever
 ↓
Relevant chunks
 ↓
LLM
 ↓
Answer
```

Phase 3 — Summarization

```
Document
 ↓
Chunks
 ↓
LLM
 ↓
Summary
```

Phase 4 — Multi-document

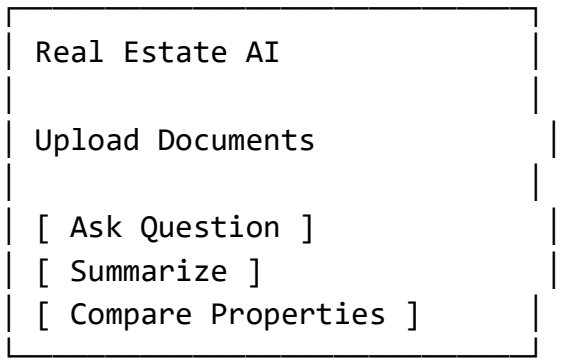
Now create:

```
Property A
Property B
```

Phase 5 — Comparison

```
Property A → Retrieve
                     \
                      → LLM → Comparison
                     /
Property B → Retrieve
```

Phase 6 — Frontend



Final structure I recommend

```
real_estate_rag/
│
├── app.py
├── requirements.txt
├── .env
│
├── documents/
│   │
│   ├── property_A/
│   │   ├── property_details.pdf
│   │   ├── rental_agreement.pdf
│   │   └── inspection_report.pdf
│   │
│   └── property_B/
│       ├── property_details.pdf
│       ├── rental_agreement.pdf
│       └── inspection_report.pdf
│
├── vector_db/
│
├── rag/                          ← COMMON RAG PIPELINE
│   ├── document_loader.py
│   ├── text_splitter.py
│   ├── embeddings.py
```

```
│   ├── vector_store.py
│   ├── retriever.py
│   └── prompts.py
│
├── features/                    ← YOUR 3 MAIN FEATURES
│   ├── qa.py                    ← 1. Document Q&A
│   ├── summarization.py         ← 2. Document Summarization
│   └── comparison.py            ← 3. Property A vs B
│
└── utils/
    └── preprocessing.py
```

So **`rag/` = how the AI finds and understands document information**, while **`features/` = what the user can do with that information**.

For development, I would start with **only one PDF and implement Step 1 → Step 6 (Document Q&A)** first. Once that works correctly, we add summarization, and only then introduce the two-property structure for comparison.